Integrantes:
Tomas Restrepo Pulgarin
Eduardo Deossa Bohorquez

Parte A

Bloque 1- Power BI y Grafana

D1. Grafana es una plataforma de código abierto que nos permite consultar, visualizar , configurar alertas y explorar nuestras métricas, registros y tazas independientemente de donde estén localizados, nos proporciona diferentes herramientas para visualizar los datos en paneles de tiempo real con gráficos y visualizaciones reveladoras. Funciona como una interfaz de visualización que traduce los datos de distintos sistemas en gráficos claras

D.2

	Power BI	Grafana
Proposito	Se usa para análisis, reportes e informes interactivos sobre datos	Visualizar métricas, logs y trazas de sistemas casi siempre en tiempo real
Usuario Tipico	Analistas de negocios, gerentes,finanzas de ventas y equipos de datos	Ingenieros DevOps/SRE, Administradores de sistemas,desarrolladores equipos de IoT
Conexión a los datos	Muchos tipos de conectores sean Excel,SQL server,Azure,APIs, etc… Los datos se importan o se consultan con DIrectQuery	Plugins de fuentes de datos sean:Prometheus,InfluxDB,loki,MySQL/PostgresSQL etc Consulta los datos directamente en su origen, sin almacenarlos
Modelo	Solido: relaciones entre tablas,medidas ,	No tiene un modelo

Semantico	como jerarquias y esquema estrella dentro del modelo(dataset)	Semanitoc propio.Cada panel ejecuta Consultas directas contra la fuente, sin relaciones a nivel de herramienta.
Actualizacion y Tiempo Real	Actualización programada streaming y DirectQuery para casos casi en tiempo real, con limitaciones.	Diseñado para tiempo real: refresco automático de paneles cada pocos segundos.
Lenguaje-DAX	Usa DAX para medidas y columnas calculadas, y Power Query (M) para transformar datos.	No usa DAX. Se usa el lenguaje de cada fuente (PromQL, SQL, LogQL, Flux) y transformaciones propias de Grafana.
alertas	Alertas básicas en dashboards (umbrales en tarjetas y medidores) y automatización con Power Automate.	Sistema de alertas avanzado: reglas multidimensionales, estados, políticas de notificación y envío a Slack, email, PagerDuty, Teams, etc.
Licenciamiento y costo	Propietario (Microsoft). Power BI Desktop es gratis; el uso colaborativo requiere licencias por usuario (Pro, Premium Per User) o capacidad (Fabric/Premium).	Código abierto (AGPL): la versión OSS es gratuita.Existen Grafana Cloud (con capa gratuita y planes de pago por uso) y Grafana Enterprise (licencia comercial).

D3. Un dashboard operativo sirve para vigilar lo que está pasando ahora mismo en la planta y reaccionar rápido, por eso se actualiza cada pocos segundos o minutos y muestra estados, alarmas y valores instantáneos.

El analítico sirve para explorar y comparar, por ejemplo ver cómo cambió la energía entre días, sitios o inversores, y permite filtrar y profundizar para encontrar causas.
El estratégico es el que usa la gerencia, con pocos indicadores y tendencias de largo plazo (mes, trimestre, año) para tomar decisiones de inversión o de operación.
Clasificaría así los KPI: la disponibilidad y las alarmas son operativos, porque alguien debe actuar en el momento, y los publicaría en Grafana por su tiempo real y su sistema de alertas. El porcentaje de datos válidos también es operativo, porque nos avisa si la telemetría está llegando bien, y también va en Grafana. La energía es analítica (energía diaria por inversor y sitio), así que va en Power BI por su modelo semántico y sus filtros. El yield y el Performance Ratio también son analíticos, porque comparan la producción contra la capacidad instalada y la irradiancia, y van en Power BI. El ahorro es estratégico, porque traduce la producción a dinero y se mira acumulado en el tiempo, y se publica en Power BI. En resumen, Grafana cubre lo que cambia minuto a minuto y Power BI lo que se analiza por día, mes o año.

D4. Una consulta de series de tiempo en Grafana con PostgreSQL debe devolver una columna de tiempo llamada time (tipo timestamp), ordenada de menor a mayor, y una o más columnas numéricas con los valores a graficar; además, el panel debe estar en formato "Time series". La macro $__timeFilter(columna) se reemplaza automáticamente por la condición del WHERE con el rango de fechas que el usuario eligió en el dashboard, así la consulta solo trae lo que se está viendo. La macro $__timeGroup(columna, '1h') agrupa los registros en intervalos de tiempo (en este caso de una hora) para poder calcular promedios, sumas o máximos por intervalo. La consulta de la potencia promedio por hora sería:

```
SELECT
$__timeGroup(ts, '1h') AS "time",
AVG(potencia_w) AS potencia_promedio
FROM silver.telemetria
WHERE $__timeFilter(ts)
GROUP BY 1
ORDER BY 1;
```

D5.En Grafana Alerting se crean reglas de alerta (alert rules), que son una consulta sobre los datos más una condición, un intervalo de evaluación y un tiempo de espera antes de disparar. Cuando la regla se activa, Grafana envía la notificación a un contact point (correo, Slack,

Teams, webhook, etc.), y las políticas de notificación deciden a quién le llega cada alerta. Las alertas de datos de Power BI son más simples: se configuran sobre un mosaico (tarjeta, medidor o KPI) de un dashboard publicado en el servicio, con un umbral de "por encima" o "por debajo", y avisan por el centro de notificaciones o por correo; para algo más elaborado hay que apoyarse en Power Automate. En resumen, Grafana evalúa los datos de forma continua y con más flexibilidad, mientras que Power BI solo mira el valor de un mosaico cuando el conjunto de datos se actualiza. Una regla útil para la planta sería: "si la potencia promedio de un inversor es menor a 1 kW durante 10 minutos seguidos, en horario de 9:00 a. m. a 3:00 p. m. y con irradiancia alta, se dispara una alerta crítica", porque eso indica que el inversor está apagado o fallando mientras debería producir. La notificaría por Microsoft Teams (con correo como respaldo) al responsable de operación de la planta.

Bloque 2 · Gobernanza de datos

G1. La gobernanza de datos es el conjunto de políticas, roles, responsabilidades y reglas que definen quién puede decidir sobre los datos, quién puede usarlos y cómo se vigila que se cumpla. Es decir, es el "gobierno" que establece las reglas del juego. La gestión de datos (data management) es todo el trabajo práctico de manejar los datos durante su ciclo de vida: almacenarlos, integrarlos, modelarlos, protegerlos y entregarlos. La calidad de datos (data quality) es una parte de esa gestión y se enfoca en medir y mejorar qué tan exactos, completos, consistentes y oportunos son los datos. En el marco DAMA-DMBOK, la gobernanza de datos va en el centro de la "rueda DAMA"(datos.gob.es, 2021) y alrededor se ubican las demás áreas, como calidad, seguridad, metadatos y arquitectura. Una forma sencilla de verlo es que la gobernanza decide y vigila, la gestión ejecuta y la calidad mide el resultado.

G2.El data owner es el dueño del dato desde el negocio: decide para qué se usa, quién tiene acceso y responde por él.
El data steward es el encargado del día a día: cuida que las definiciones, las reglas y la calidad se cumplan (por ejemplo, qué significa exactamente "energía diaria" o "dato válido").
El data custodian es el responsable técnico de guardar y proteger los datos, es decir, de la infraestructura, las copias de seguridad y los accesos.
En el proyecto propongo que el product owner sea el data owner, porque define qué se necesita y prioriza el uso de los datos de la planta. El BI analyst y el data modeler harían de

data stewards: el analista define y valida los KPI, y el modelador mantiene las definiciones y la estructura de las tablas (dimensiones y hechos). El data engineer sería el data custodian, porque administra PostgreSQL, los pipelines de carga, los respaldos y los permisos.

G3.El contrato de datos es un acuerdo escrito entre quien produce el dato (la fuente) y quien lo consume (nuestro tablero), y sirve para que ambos sepan qué esperar. Debe incluir el esquema (nombres y tipos de cada campo), las unidades (kW, kWh, °C, W/m²), la frecuencia de llegada (cada 5 minutos), el responsable de la fuente y a quién avisar, y los niveles de calidad exigidos (porcentaje mínimo de datos completos, rangos válidos, tiempo máximo de retraso), además de la versión y cómo se avisan los cambios. Si la fuente cambia un campo sin avisar, el tablero puede romperse de forma visible (error al actualizar por una columna que ya no existe) o, peor, de forma silenciosa, mostrando ceros, vacíos o valores en otra escala (por ejemplo W en vez de kW), lo que haría que los KPI queden mal sin que nadie lo note. Por eso conviene validar el esquema al entrar a la capa Silver, mandar los registros dudosos a cuarentena y tener una alerta que avise cuando algo no cumple el contrato.

G4 La RLS se configura definiendo roles y reglas con expresiones de filtro DAX(Microsoft, s. f.-a; DataCamp, s. f.) en Power BI Desktop; luego se publica el modelo y se asignan los usuarios a cada rol en el servicio de Power BI. Por ejemplo, para que un responsable solo vea su sitio se crea un rol llamado "Responsable_Sitio" con un filtro sobre la tabla dim_sitio. En su forma estática sería [sitio_nombre] = "Sitio Norte"; en la dinámica, que es mejor, se usa una tabla de correspondencia y el filtro [correo_responsable] = USERPRINCIPALNAME(), para que cada persona vea automáticamente solo lo suyo. Esto se prueba con la opción "Ver como" y funciona para quienes entran con rol de Viewer en el área de trabajo. En Grafana el equivalente es más general: se usan organizaciones (que separan completamente dashboards y fuentes de datos), equipos de usuarios y roles (Viewer, Editor, Admin), con permisos asignados por carpeta o por dashboard. La diferencia importante es que Grafana controla quién ve qué dashboard, pero no filtra filas por usuario; si se necesita eso hay que resolverlo en la base de datos, por ejemplo con RLS nativa de PostgreSQL o con una fuente de datos distinta por grupo.

G5. La ley define el dato personal como cualquier información vinculada(Congreso de la República de Colombia, 2012, art. 3, lit. c; Superintendencia de Industria y Comercio [SIC], 2015) o que pueda asociarse a una o varias personas naturales determinadas o determinables.

Los datos de SolarBI (energía, potencia, irradiancia, temperatura, estado de los inversores) son datos técnicos de una planta y no se refieren a ninguna persona, así que en principio no son personales y la ley no aplica a ellos. Eso cambiaría si el tablero mostrara qué operador atendió cada alarma, porque ahí el dato queda asociado a una persona identificada y pasaría a ser un dato personal: habría que tener autorización del titular, informarle para qué se usa (finalidad), pedir solo lo necesario, restringir quién lo ve (por ejemplo con RLS) y protegerlo con medidas de seguridad. Además, usarlo para medir el desempeño de alguien exige más cuidado. Nuestra postura como equipo sería recolectar solo los datos que realmente se necesitan, usar identificadores como el rol o el turno en lugar del nombre cuando se pueda, dar acceso por roles, no compartir información de personas sin autorización y documentar para qué se usa cada dato.

Bloque 3. Automatizacion de procesos ETL

E1. En ETL los datos se extraen, se transforman en un servidor intermedio y solo después se cargan al destino, mientras que en ELT primero se cargan crudos al destino y ahí mismo se transforman aprovechando la potencia de la base de datos. En el proyecto la extracción desde los inversores y sensores deja los datos tal cual llegan en Bronze (carga cruda). En Silver se limpian, se corrigen tipos y unidades, se quitan duplicados y se validan. En Gold se construye el modelo dimensional y los KPI, como fact_energia_dia. Como cargamos primero y transformamos con SQL dentro de PostgreSQL, el proyecto se parece más a un ELT. Sobre si Power Query es una herramienta ETL, diría que sí en parte: tiene conectores para extraer, pasos para transformar y carga el resultado al modelo de Power BI. Pero es una herramienta ligera de preparación de datos para el analista, no un orquestador de pipelines de producción (no maneja bien dependencias, reintentos ni cargas incrementales complejas), y cuando hay query folding se comporta más como ELT porque delega el trabajo a la base(Microsoft, s. f.-b) .

E.2 En una carga completa (full load) se borra y se vuelve a cargar toda la tabla; es simple, pero cada vez es más lenta y costosa a medida que crecen los datos. En una carga incremental solo se procesa lo nuevo o lo que cambió desde la última ejecución, normalmente usando una fecha como marca. Una carga es idempotente cuando se puede ejecutar varias veces con los mismos datos y el resultado siempre es el mismo, sin duplicados. En fact_energia_dia esto se

logra porque la clave primaria (fecha_key, dispositivo_key) impide que haya dos filas para el mismo día y dispositivo, y el UPSERT inserta la fila si no existe o la actualiza si ya existe:

```
INSERT INTO gold.fact_energia_dia (fecha_key, dispositivo_key, energia_kwh)
VALUES (20260310, 5, 124.7)
ON CONFLICT (fecha_key, dispositivo_key)
DO UPDATE SET energia_kwh = EXCLUDED.energia_kwh;
```

E3.

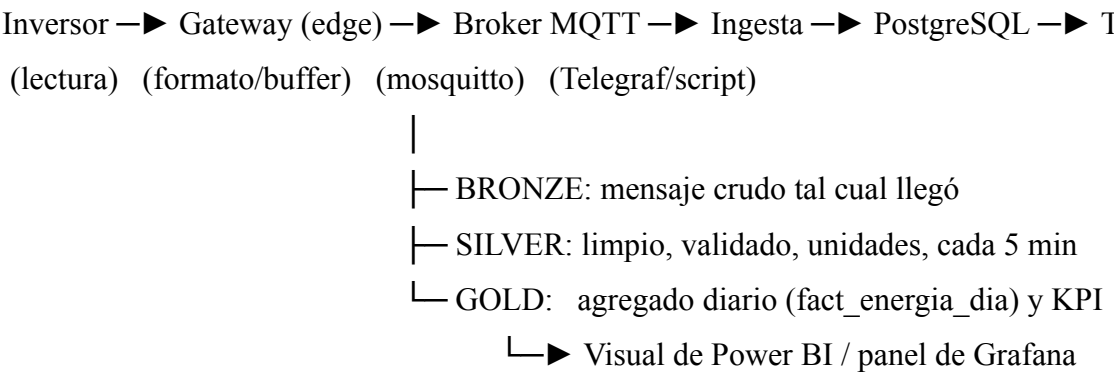
	Cron / Programador de tareas de Windows	Apache Airflow	Actualización programada de Power BI
Qué es	Programador del sistema operativo que ejecuta un comando o script a una hora definida.	Plataforma de orquestación donde los flujos se definen como DAG (tareas con dependencias) en Python.	Función del servicio de Power BI que vuelve a cargar un modelo publicado en horarios definidos.
Qué automatiza	Un script suelto (por ejemplo, un Python que carga datos a PostgreSQL).	Pipelines completos de varias tareas, con orden, reintentos, registros y monitoreo.	Solo la actualización del conjunto de datos de Power BI desde sus fuentes.
Cuándo conviene	Tareas simples e independientes, con poca infraestructura.	Flujos con varios pasos y dependencias (Bronze → Silver → Gold), que necesitan control y trazabilidad.	Cuando el modelo ya está en Power BI y solo hay que refrescarlo, sin lógica de pipeline.

E4. El query folding es la capacidad de Power Query de traducir los pasos de transformación a una sola consulta nativa (en nuestro caso SQL) y enviarla a PostgreSQL para que la base haga el trabajo, en vez de traer todos los datos y procesarlos en Power BI. Mejora el rendimiento porque se traen solo las filas y columnas necesarias, se aprovechan los índices de la base y se reduce la memoria y el tiempo de actualización. Para verificar si un paso se está plegando se hace clic derecho sobre el paso en Power Query y se elige "Ver consulta nativa"; si la opción aparece deshabilitada, el plegado se rompió en ese paso o antes. También se

pueden usar los indicadores de plegado de los pasos aplicados y el plan de consulta. Conviene dejar al final los pasos que rompen el plegado, como las funciones personalizadas o Table.Buffer

Bloque 4 · Internet de las Cosas (IoT)

I1. La arquitectura tiene cinco capas. La de dispositivo la forman el inversor, el medidor y los sensores de irradiancia y temperatura, que generan las lecturas. La de gateway o edge es un pequeño equipo cerca de la planta (por ejemplo un datalogger o una Raspberry Pi) que lee los dispositivos (por ejemplo por Modbus), les da formato y guarda temporalmente si se cae la red. La de red o broker es el broker MQTT, que recibe los mensajes y los reparte a quien esté suscrito. La de almacenamiento es la base de datos (PostgreSQL) donde se guarda todo. La de aplicación es donde se visualiza (Grafana y Power BI). El recorrido de una lectura sería:



I2.MQTT se basa en tres roles: el publicador, que es el dispositivo que genera los datos; el broker, un servidor que recibe los mensajes y los enruta según las suscripciones; y el suscriptor, la aplicación que se interesa por un tema. Esto se llama modelo publish/subscribe y permite que el inversor no tenga que saber quién consume sus datos. El topic es el "tema" o dirección del mensaje, escrito por niveles separados con /, y el suscriptor puede usar comodines (+ para un nivel, # para todos los siguientes). Los niveles de QoS indican qué tan segura es la entrega: QoS 0 envía una sola vez sin confirmación (puede perderse), QoS 1 garantiza que llegue al menos una vez (puede duplicarse) y QoS 2 garantiza que llegue exactamente una vez, a cambio de ser más lento. Para la telemetría cada 5 minutos usaría QoS 1, apoyada en una carga idempotente que absorba duplicados, y para las alarmas

también QoS 1 o 2. La jerarquía de topics propuesta sería pascualbravo/solar/<sitio>/<dispositivo>/<variable>, por ejemplo pascualbravo/solar/bloque-a/inversor01/potencia_ac o pascualbravo/solar/bloque-a/inversor01/estado. Con ella se puede suscribir a todo un sitio con pascualbravo/solar/bloque-a/# o a una sola variable de todos los inversores con pascualbravo/solar/+/+/potencia_ac.

I3.Mosquitto es el broker MQTT que recibe los mensajes de los dispositivos y los reparte. Telegraf es un agente que se suscribe a los topics, convierte los mensajes y los escribe en la base de datos. La base de series de tiempo (InfluxDB o TimescaleDB) está optimizada para guardar y consultar datos ordenados por tiempo, con compresión y agregaciones rápidas. Grafana se conecta a esa base y muestra los paneles y las alertas. PostgreSQL encaja de dos formas: como la base que guarda las capas Bronze, Silver y Gold, y, con la extensión TimescaleDB (que es una extensión de PostgreSQL), como base de series de tiempo, con lo cual una sola base puede servir tanto a Grafana como a Power BI.

I4. El procesamiento por lotes (batch) junta los datos y los procesa cada cierto tiempo (por ejemplo, cada noche), mientras que el flujo continuo (streaming) procesa cada dato casi en el momento en que llega. Al agregar la telemetría de 5 minutos en una tabla diaria se gana velocidad, porque hay muchas menos filas, las consultas y el modelo de Power BI son más livianos, los KPI son más estables y se ahorra almacenamiento. Pero se pierde detalle: ya no se ven los picos de potencia, la curva del día, las caídas breves, el efecto de las nubes ni el momento exacto de una falla, y no se puede reconstruir el dato original si no se conservó. Por eso conviene mantener los 5 minutos en Silver y solo agregar en Gold. Grafana atiende mejor la granularidad fina y el tiempo real (5 minutos o menos), y Power BI atiende mejor la granularidad diaria o mayor, que es el análisis por lotes.

Bloque 5 · Control de versiones

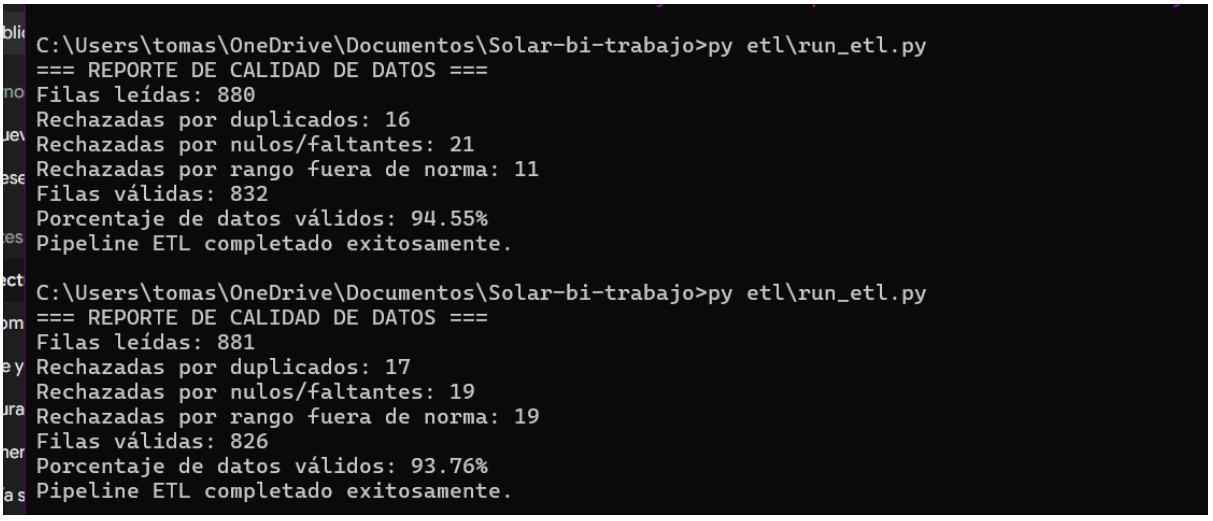
V1.Git es un sistema de control de versiones distribuido que se instala en nuestro computador y guarda el historial de cambios del código, mientras que GitHub es una plataforma web que aloja repositorios Git y agrega herramientas de colaboración como pull requests(Chacon y Straub, 2014; freeCodeCamp, s. f.) , issues y automatización. Un repository es la carpeta del proyecto con todo su historial. Un commit es una "foto" del estado del código en un momento dado, con un mensaje que describe el cambio, una branch (rama) es una línea de desarrollo paralela para trabajar sin afectar la rama estable, y un pull request es la solicitud formal de

integrar una rama en otra, con revisión incluida. Un archivo .pbix es binario (un paquete comprimido), por lo que Git no puede mostrar qué cambió entre versiones, no se pueden resolver conflictos cuando dos personas lo editan y el repositorio crece con cada versión guardada. El formato Power BI Project (.pbip) resuelve esto porque guarda el modelo y los informes como carpetas con archivos de texto, lo que permite ver diferencias, revisarlas en un pull request y combinar el trabajo de varias personas. Un dashboard de Grafana se versiona exportándolo como JSON (o gestionándolo con provisioning o con la API) y guardando ese archivo en el repositorio, y Grafana además conserva su propio historial de versiones de cada dashboard.

Pruebas B

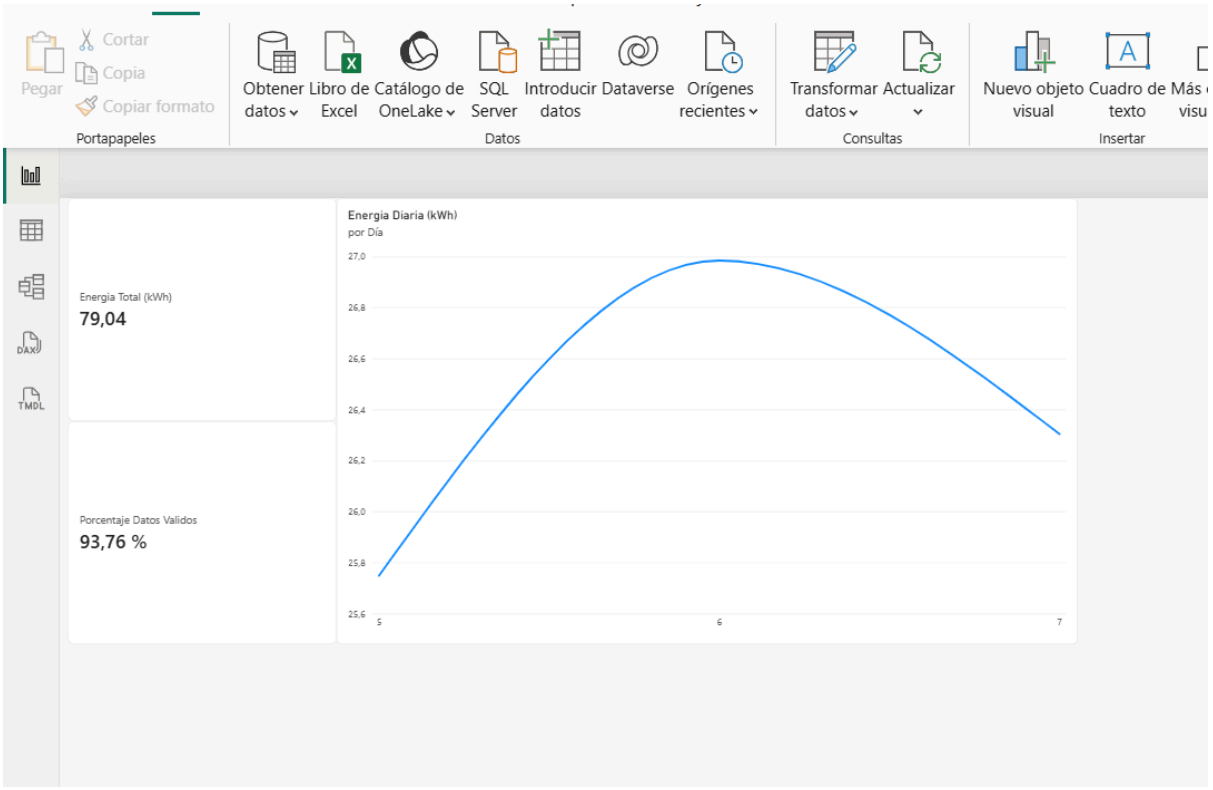
En esta primera imagen mostramos la pantalla de la consola de comandos de Windows al momento de ejecutar nuestro script principal run_etl.py. Aquí se evidencia todo el proceso de lectura y limpieza de la telemetría. El programa tomó el archivo plano telemetria.csv con un total de 881 filas y le aplicó las reglas de filtro que programamos en Python.

Se observa que el script descartó 17 filas por estar duplicadas, 19 por tener valores nulos o incompletos y 19 más por estar fuera del rango normal de operación. Al final, quedaron 826 filas válidas, lo que nos da un 93.76% de datos limpios. El mensaje final *" Pipeline ETL completado exitosamente"* confirma que la información se guardó de forma correcta y sin errores dentro de la base de datos de PostgreSQL.

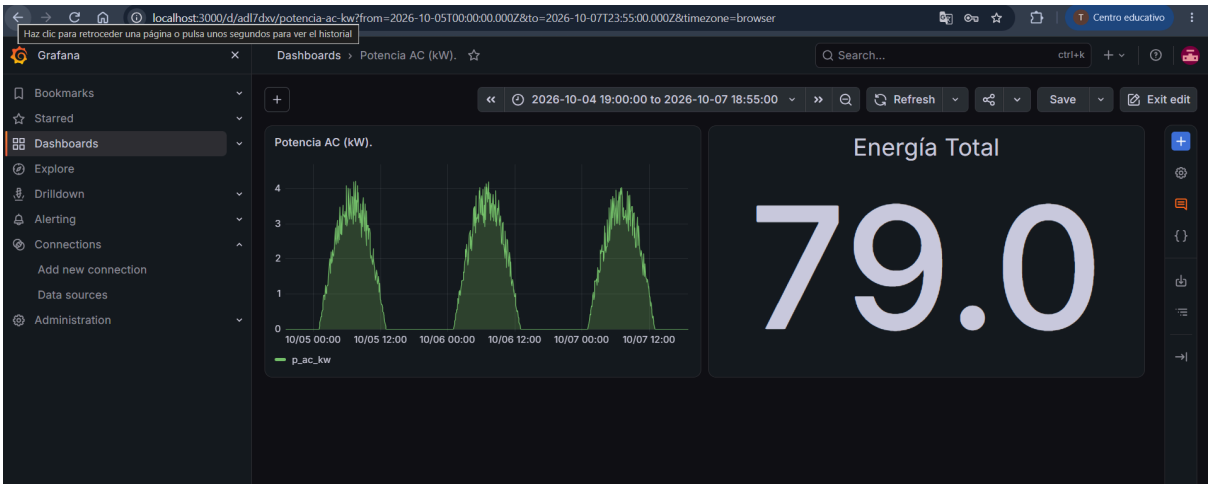


En la segunda captura se muestra el reporte desarrollado en Power BI Desktop a partir de la tabla de resumen diario guardada en PostgreSQL, tal como lo solicitaba la guía del taller. Siguiendo las indicaciones de la actividad, se ubicaron a la izquierda las dos tarjetas de indicadores requeridas: la Energía Total acumulada (79,04 kWh) y el Porcentaje de Datos Válidos (93,76%), ambas calculadas mediante las medidas DAX solicitadas. Asimismo, a la

derecha se construyó el gráfico de líneas que requería la guía para representar la evolución de la energía producida día a día entre el 05, 06 y 07 de octubre.



En la tercera imagen mostramos el dashboard configurado en la interfaz de Grafana (localhost:3000), cumpliendo con los requerimientos fijados para la parte práctica de monitoreo operativo. De acuerdo con las instrucciones de la guía, en el panel de la izquierda se configuró la consulta en formato de serie temporal (*Time Series*) para mostrar la potencia cada 5 minutos, donde se observan claramente las tres campanas verdes de generación diaria. En el panel de la derecha se integró la tarjeta de tipo *Stat* para indicar la Energía Total (79.0 kWh), comprobando que los datos consultados directamente desde PostgreSQL responden a lo exigido en el taller y coinciden con Power BI.



Parte 3 Enlace git hub:https://github.com/Tomixddxrt/solarbi-Restrepo-Deossa

Fuentes:

- **Microsoft Learn, "Seguridad a nivel de fila (RLS) con Power BI": https://learn.microsoft.com/power-bi/enterprise/service-admin-rls**
- **Microsoft Learn, "Plegado de consultas en consultas nativas (Power Query)": https://learn.microsoft.com/power-query/native-query-folding**
- **Superintendencia de Industria y Comercio (SIC), Concepto 15-283428 sobre la Ley 1581 de 2012 y la definición de dato personal: https://sedeelectronica.sic.gov.co/sites/default/files/normatividad/Concepto_15-283428.pdf**
- **datos.gob.es (Gobierno de España), "El potencial uso de la metodología de DAMA en la gestión de los datos abiertos": https://datos.gob.es/es/blog/el-potencial-uso-de-la-metodologia-de-dama-en-la-gestion-de-los-datos-abiertos**
- **Pro Git, versión en español (git-scm.com): https://git-scm.com/book/es/v2**
- **Tractian, glosario "MQTT (Message Queuing Telemetry Transport)": https://tractian.com/es/glosario/mqtt-message-queuing-telemetry-transport**
- **DataCamp en español, "Seguridad a nivel de fila (RLS) de Power BI": https://www.datacamp.com/es/tutorial/power-bi-row-level-security-rls**
- **freeCodeCamp en español, "El manual esencial de Git": https://www.freecodecamp.org/espanol/news/el-manual-esencial-de-git/**